

Итоги практики 6

✔ Большинство хорошо справились

Задание оказалось сложнее чем выглядело — и вы с ним справились. Это не само собой разумеется.

ⓘ Видно погружение в инструмент

Не просто «скопировал сценарий», а разобрались как работает n8n — nodes, executions, credentials, data flow.

💡 Алгоритмическое мышление — это база

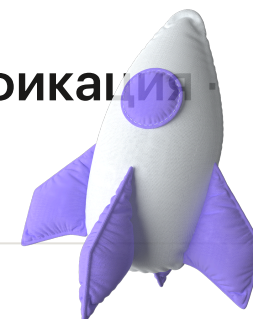
Workflow — это алгоритм. Условие, ветвление, цикл. Те, кто мыслит алгоритмами, собирают сценарии быстрее и чище.

Хороший workflow — это хороший алгоритм. Чёткие входы и выходы на каждом шаге, минимум лишних переходов, понятная логика ветвления.

Что берём в сегодняшнюю практику: тот же подход — сначала схема на бумаге или в голове, потом n8n. Не наоборот.

Сегодня идём дальше:

GitHub Issues · AI-классификация
Sub-workflow · Data table





Практика 8

Поддержка пользователя с помощью AI инструментов



GitHub Issues · n8n · 3 workflow · Gmail



План занятия



01

GitHub Issues

CJM, типы, жизненный цикл, лейблы, масштаб, GitHub native vs n8n

~20 мин

02

Workflow 1

Классификация Issues — триггер, first-time-contributor, AI-классификатор, Switch, лейблы

~25 мин

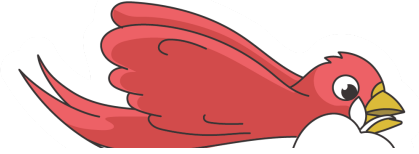
03

Практика + ДЗ

4 задания по нарастающей. Workflow 2, 3, 4 — домашнее задание

~45 мин

Домашнее задание закрывает полный цикл: WF2 (PR → in-progress) · WF3 (merge → close + AI summary) · WF4 (еженедельная статистика → Gmail)



1. GitHub Issues

1. Что такое Issue и типы обращений
2. CJM: жизненный цикл
3. Лейблы, assignee, milestone
4. Проблема масштаба и автоматизация



Что такое GitHub Issue

Единица коммуникации в репозитории — задача, баг, идея или вопрос



Issue

Задача, баг, идея, вопрос. Обсуждение + статус + метаданные. Может жить месяцами.
Создаёт: кто угодно



Pull Request

Изменение кода. Привязан к ветке.
Закрывается при мерже. Создаёт:
разработчик



Discussion

Открытый разговор. Нет статуса и assignee.
Не привязан к задаче. Создаёт: сообщество

✓ **Факт:** Issues существуют не только для багов — это основной канал общения между пользователями и командой

Три типа Issues



Bug

Что-то работает не так. Есть шаги воспроизведения, expected vs actual.
Версия и окружение.

App crashes on large file

Steps:

1. Open file > 10MB
 2. Press Ctrl+F
- crash



Feature request

Новая возможность. Описание желаемого поведения и обоснование зачем. Часто без технических деталей.

Add vim keybindings

Would improve productivity
for vim users daily



Task / Question

Вопрос о существующем функционале.
Часто нет воспроизведения. Решается

комментарием или ссылкой на docs.

How to configure
custom theme colors?
Can't find in docs.



CJM: жизненный цикл Issue



👤 **Автор Issue:** Описывает проблему → отвечает на уточнения → подтверждает fix

🔗 **Мейнтейнер:** Классифицирует → вешает лейблы → назначает → закрывает

△ **n8n / бот:** Автоматически — лейблы → комментарии → уведомления → статистика

Метаданные Issue: лейблы, assignee, milestone

Типы лейблов

bug feature request task documentation
triage-needed in-progress needs-info
critical high medium low

1 Assignee

Кто работает над задачей. Может быть несколько человек.

2 Milestone

К какому релизу или спринту относится. Помогает планировать.

3 Projects / Kanban

Статус на доске: Todo → In Progress → Done. GitHub умеет двигать автоматически.



Проблема масштаба

14 000+
открытых
225 000+ закрытых

Issues в репозитории VS Code прямо сейчас

1 5–10 новых Issues каждый день

Нужно прочитать, классифицировать, ответить

2 Один человек не справится

Команда ротирует inbox tracker понедельно

Без автоматизации

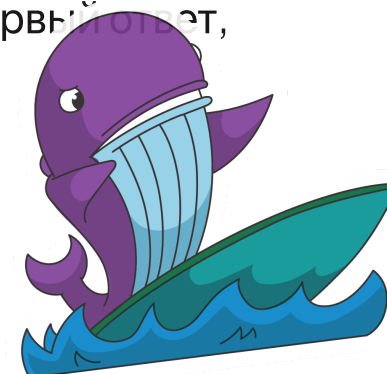
Issues без ответа неделями

Нет единообразия в лейблах

Дубликаты не отслеживаются

- Авторы не знают статус задачи

✓ **Решение:** Автоматизируем рутину — классификация, лейблы, первый ответ, уведомления



Что GitHub умеет сам — и где нужен n8n

GITHUB ИЗ КОРОБКИ

- ✓ **closes #123 в PR body**
GitHub закрывает Issue при мерже автоматически
- ✓ **first-interaction action**
Приветствие нового участника без кода
- ✓ **GitHub Projects automation**
Статусы на канбан при изменении PR
- ✓ **Stale bot**
Закрытие старых Issues без активности

ЗДЕСЬ НУЖЕН N8N

- **AI-классификация Issue**
GitHub не знает что такое bug vs feature
- **in-progress по коммиту**
GitHub не слушает паттерны в сообщениях
- **AI summary при мерже**
Генерация текста требует LLM
- **Еженедельная статистика**
Агрегация + отправка email = n8n

2. Workflow 1: автоматизация разбора Issues

1. Схема и связь с практикой 6
2. Демо: триггер GitHub Issues
3. Демо: first-time-contributor
4. Демо: AI классификатор → Switch → лейблы

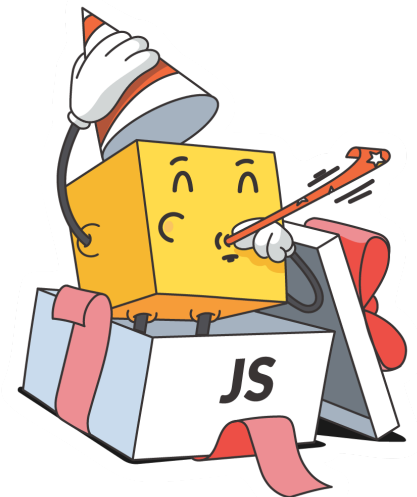


Схема Workflow 1



Из практики 6 — переиспользуем

Webhook-триггер (GitHub)

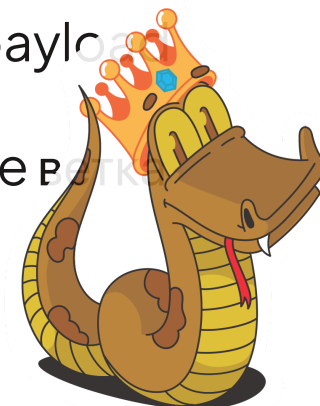
Basic LLM Chain + Switch node

HTTP Request к GitHub API

Data table

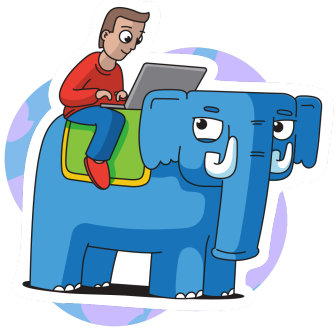
Новое сегодня

- Issues вместо PR как триггер
- first-time-contributor логика
- author_association из payload
- Эскалация: needs-triage в





Let's make it...



Демо: триггер и структура payload

1 Webhook в n8n

GitHub → n8n, events: `issues`, фильтр
`action = "opened"`

2 Ключевые поля payload

`issue.number` · `issue.title` · `issue.body`
· `issue.user.login` ·
`issue.author_association`

3 Проверяем в Executions

Создаём тестовый Issue в репозитории,
смотрим вход ноды

Факт: `author_association` — встроенное поле GitHub. `NONE` означает первый вклад в репозиторий.

```
// Payload от GitHub
{
  "action": "opened",
  "issue": {
    "number": 42,
    "title": "App crashes on...",
    "body": "Steps to reproduce",
    "user": {
      "login": "john_doe"
    },
    "author_association": "NONE"
  },
  "repository": {
    "name": "my-project",
    "owner": {
      "login": "org-name"
    }
  }
}
```

}

}

Демо: AI классификатор → Switch → лейблы

Switch node — четыре ветки

bug label: **bug** + комментарий с шаблоном

task label: **task** + комментарий с шаблоном

feat label: **feature-request** + комментарий

? label: **needs-triage** — без комментария

```
// Промпт в Basic LLM Chain
Classify this GitHub Issue.
Title: {{ $json.issue.title }}
Body: {{ $json.issue.body }}
```

Return ONLY JSON:

```
{ "type": "bug",
  "confidence": 0.91 }
```

```
Types: bug | task |
       feature | unknown
```

Важно: **confidence** < 0.8 → ветка
unknown → лейбл **needs-triage** , без
комментария

Structured Output Parser

БЕЗ ПАРСЕРА — РУЧНОЙ JSON

```
// Наш промпт явно просит JSON:
Return ONLY JSON:
{ "type": "bug",
  "confidence": 0.91 }

// Модель отвечает... примерно так:
```json
{ "type": "bug", "confidence": 0.91 }
```

// Или так:
Sure! Here's the result:
{ "type": "bug" ... }

// Set node разбирает JSON вручную
JSON.parse($json.output)
```

С ПАРСЕРОМ — АВТОМАТИЧЕСКИ

1 Описываем схему

Задаём поля `type` (string) и `confidence` (number) в настройках парсера

2 Парсер дописывает промпт

n8n автоматически добавляет в конец промпта инструкцию про формат JSON и описание схемы — вы это не видите явно

3 Результат — готовый объект

На выходе ноды уже `$json.type` и `$json.confidence` — без Set node и без `JSON.parse`

Осторожно: парсер повторяет запрос если модель вернула невалидный JSON. Это

незаметно, но увеличивает стойкость и латентность.



Демо: first-time-contributor

✓ first-time-contributor

Описывает **автора** — человек
НОВЫЙ

Определяем через

```
author_association = "NONE"
```

Сигнал: уделить больше внимания,
поздороваться

Ссылка на CONTRIBUTING.md в
приветствии

✗ good-first-issue — не то

Описывает **задачу** — она простая
для новичка

Вешает мейнтейнер **вручную** —
оценка сложности

Нельзя определить автоматически

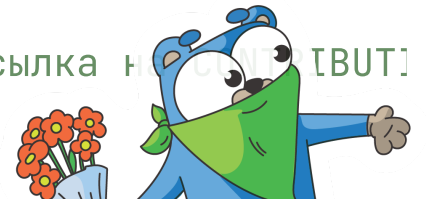
Автоматическая постановка —
ложное обещание

```
// If node: первый контрибьютор?
```

```
"author_association" == "NONE"
```

→ POST /labels ["first-time-contributor"]

→ POST /comments "Привет @{{ user.login }}! Рады вашему Issue. Вот ссылка на ваш первый Issue. [BUT]"



Запустили — что получилось?

Создайте несколько тестовых Issues. Где агент ошибается?

“Какие Issues классифицированы неверно?”

> Ответ

Пограничные случаи: «не могу войти» — это bug или task? Расплывчатые описания без конкретики обычно падают в unknown.

“Почему confidence низкий?”

> Ответ

Короткий или нечёткий body, или Issue содержит элементы нескольких типов одновременно.

“Как улучшить промпт?”

> Ответ

Few-shot: добавить по одному примеру каждого типа прямо в промпт. Это один из самых эффективных способов улучшить точность.

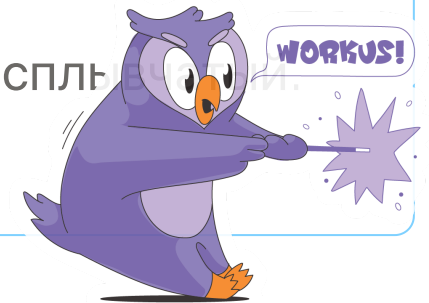
“Что делать с Issues не по теме?”

> Ответ

Добавить ветку off-topic в Switch: закрыть Issue с комментарием-объяснением.

⚡ МИНИ-ПРАКТИКА · 3 МИН

Создайте Issue с заголовком «Иногда не работает» — намеренно расплывчатый заголовок.
Посмотрите, что вернёт классификатор и с каким confidence.



4. Самостоятельная работа

1. Задание 1 — базовый Workflow 1
2. Задание 2 — first-time + эскалация
3. Задание 3 — Workflow 2: PR → in-progress
4. Задание 4 — Sub-workflow
5. ★ Задание 5 — несколько Issues в PR



4 задания

01

Workflow 1

Базовый WF1

Развернуть сценарий. Проверить лейблы bug / task / feature для тестовых Issues.

02

Workflow 1

first-time + эскалация

Ветка first-time-contributor с приветствием.
Ветка unknown — needs-triage без комментария.

03

Sub-workflow

Sub-workflow

Вынести AI-классификатор в отдельный сценарий. Подключить через Execute Workflow node.

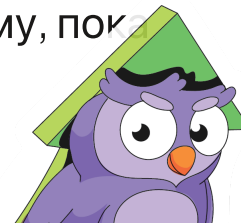
04

Data table

Дедупликация

Сохранять ID Issue в data table. Не отвечать повторно на уже обработанные Issues.

Порядок важен: каждое задание опирается на предыдущее. Не переходи к следующему, пока текущее не работает в Executions.



01 Задание 1 — базовый Workflow 1

1 Скачать и импортировать

JSON-сценарий WF1 импортировать в n8n, настроить GitHub webhook на events: issues

2 Создать тестовые Issues

По одному Issue каждого типа: явный баг, вопрос, запрос фичи

3 Проверить в Executions

Убедиться что лейблы проставлены корректно и агент не ошибся в классификации

Подсказка: Открой Executions → смотри поле `type` и `confidence` в ответе LLM. Если `confidence < 0.8`, задача уходит в needs-triage.



02 Задание 2 — first-time-contributor + эскалация

1 Ветка first-time-contributor

`author_association = "NONE"` → лейбл + приветственный комментарий со ссылкой на CONTRIBUTING.md

2 Ветка unknown (эскалация)

`confidence < 0.8` → лейбл `needs-triage`, комментарий НЕ публикуется

3 Тестирование

Тест 1: Issue с нового аккаунта. Тест 2: расплывчатый Issue без конкретики

Подсказка: Для теста first-time-contributor создай тестовый аккаунт на GitHub или попроси коллегу открыть Issue.



03 Задание 3 — Sub-workflow

1 Создать Sub-workflow: классификатор

Новый сценарий только с AI-логикой. Execute Workflow trigger принимает `title` и `body`, возвращает `type` и `confidence`.

2 Подключить в основной сценарий

Заменить Basic LLM Chain на Execute Workflow node. Убедиться что результат классификации передаётся корректно.

3 Проверить что ничего не сломалось

Создать Issue всех типов — поведение должно быть идентичным до рефакторинга.

Зачем: Sub-workflow — главный паттерн переиспользования в n8n. Классификатор может быть вызван из WF2, WF3 и любого другого сценария без дублирования кода.



04 Задание 4 — Дедупликация через data table

1 Создать таблицу processed_issues

В Data Tables n8n: поля `issue_id` , `author` , `created_at` .

2 Проверить перед ответом

При получении Issue — ищем `issue.number` в таблице. Если найден — завершаем сценарий без ответа.

3 Записать после ответа

Если Issue новый — обрабатываем как обычно и добавляем запись в таблицу.

Тест: Закрой Issue и открой снова — бот не должен ответить повторно. Проверь таблицу в n8n — запись должна быть.

Домашнее задание

Часть 1 • 4 балла

WF1

Базовый Workflow 1

Webhook на `issue opened`

AI-классификатор: `bug / task / feature`

Switch → лейбл для каждого типа

Ветка `unknown` : лейбл `needs-triage`
без комментария

- Sub-workflow для классификатора

Часть 2 • 4 балла

cron пн 9:00

Еженедельная статистика → Gmail

Cron-триггер: `0 9 * * MON`

4 GitHub API запроса за неделю

Code node: группировка по типу и автору

- Gmail node → HTML-письмо с таблицей

Diff недели + AI-аннотация

Сохранять статистику в Data Table

Вычислять Δ к прошлой неделе

Basic LLM Chain генерирует
аннотацию изменений

 Блок «AI-комментарий недели» в
письме





Ваши вопросы?





Вы большие молодцы!

Серьёзно — работать с AI-инструментами
и выстраивать алгоритмы с нуля это
непросто. Вы справились!

